

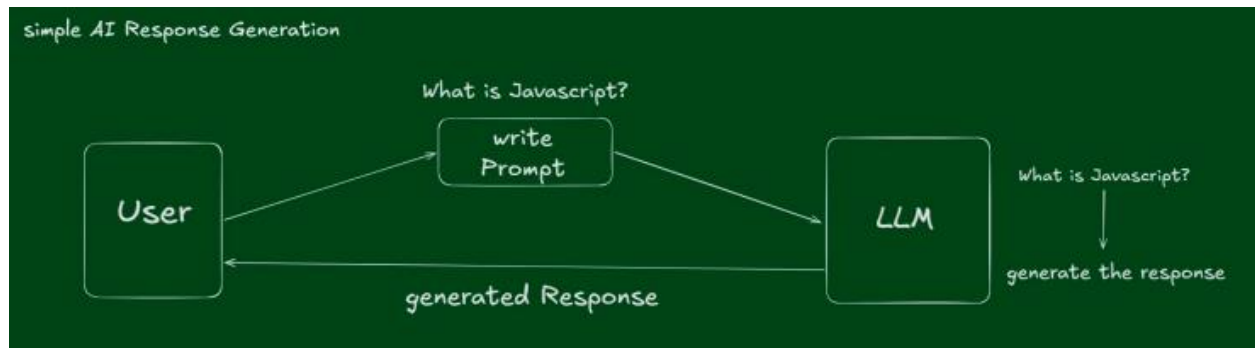
Advance Backend Part 2

LLM: stands for Large Language Model.

It is an AI model that train on massive amount of text data to understand and generate human language.

Example:

openai GPT models, Anthropic Claude, Google Gemini, Meta Llama



Simple ai based Application/website

You ask: "what is ai"

The LLM predicts next tokens and generate response/answer.

Limitations of LLM: An LLM by itself:

1. does not access tools automatically.
2. Does not remember long-state reliably.
3. Does not take action in the real world unless connected with system.

It is mainly a test intelligence engine.

Build simple ai-chatbot using gemini llm api (because its free)

Level4/ //setup basic express server npm init --- app.listen() etc.

search on google 'google gemini api' & open ' <https://ai.google.dev/gemini-api/docs> '

flow of chatbot task: (get gemini api for google ai studio) – (copy getstarted code mentioned in doc)

We use google gemini sdk for our simple chatbot because SDK give us suggestion while rest api not.

npm install @google/genai

```
import { GoogleGenAI } from "@google/genai";
const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });

app.post("/chat", async (req, res) => {
  const { input } = req.body;
  const interaction = await ai.interactions.create({
    model: "gemini-3.5-flash",
    input,
  });
  res.status(200).json({ output_text: interaction.output_text });
});
```

Example Agent Workflow

User:

"Find the cheapest flight to Delhi and email me details."

Agent:

1. Understands request using LLM
2. Searches flights
3. Compares prices
4. Chooses best option
5. Sends email
6. Returns result

That entire workflow = AI Agent.

LLM + Realtime data + Memory + action

AI Agent

Lang Chain
Lang Graph

Difference Between LLM and Agent

Feature	LLM	Agent
Generates text	✓	✓
Uses tools/APIs	✗	✓
Makes decisions	Limited	✓
Multi-step tasks	Weak	Strong
Memory	Minimal	Persistent
Autonomous actions	✗	✓

Popular Agent Frameworks

Developers commonly use:

- [LangChain ↗](#)
- [LangGraph ↗](#)
- [LlamaIndex ↗](#)
- [CrewAI ↗](#)
- [AutoGen ↗](#)

What is LangChain?

LangChain is a framework used to build:

- AI chatbots
- RAG systems
- AI agents
- Tool-calling apps
- Multi-step AI workflows

It helps connect:

LLM + Memory + Tools + Vector DB + APIs

LangGraph

LangGraph is used for:

- Stateful AI agents
- Memory
- Multi-agent systems
- Workflow orchestration

This is production-level AI engineering.

1. How to Call LLM in Langchain: <https://docs.langchain.com/oss/javascript/integrations/chat/groq>

Level4/index.js npm install @langchain/groq @langchain/core //install langchain pkg for grok

```
import express from "express";
import dotenv from "dotenv";
import { ChatGroq } from "@langchain/groq";

dotenv.config();

const app = express();
app.use(express.json());
const port = process.env.PORT || 4000;

const llm = new ChatGroq({
  model: "openai/gpt-oss-120b",
  temperature: 0.7, //creativity of model-response: 0-> strict response, temp>0 -
> creativity & long response
  maxTokens: 100, //generate response with maxtoken range.
  maxRetries: 2,
});

//without langchain CHATBOT
// const ai = new GoogleGenAI({ apiKey: process.env.GOOGLE_API_KEY });

// app.post("/chat", async (req, res) => {
//   const { input } = req.body;
//   const interaction = await ai.interactions.create({
//     model: "gemini-3.5-flash",
//     input,
//   });
//   res.status(200).json({ output_text: interaction.output_text });
// });

// with langchain CHATBOT
app.post("/chat", async (req, res) => {
  const { input } = req.body;

  const aiMsg = await llm.invoke(input);
  res.status(200).json({ response: aiMsg.content });
});

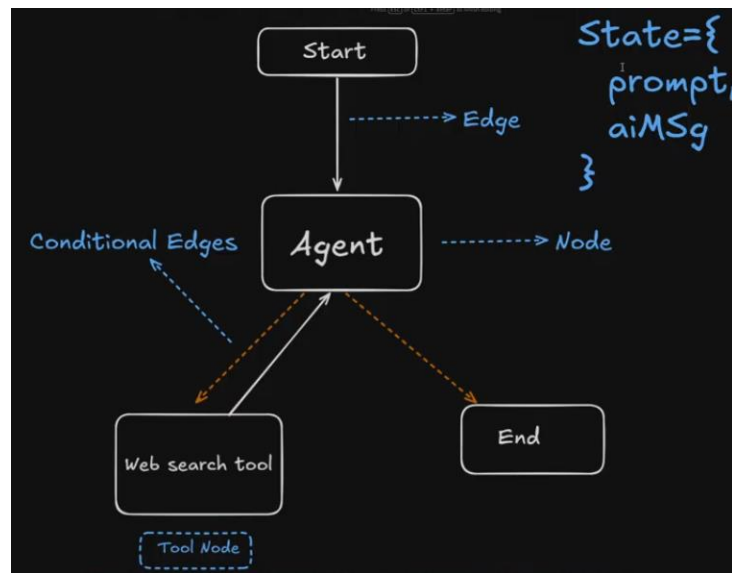
app.listen(port, () => {
  console.log(`Server is Listen on ${port}`);
});
```

Note: here we create .env file where we add cridentials like GROK_API_KEY but not config api in index.js(langchain object), why: because langchain docs says if you write api key with default name 'GROQ_API_KEY' then langchain framework auto detect & config it internally, so no need to manually config it.

2. Implement AI-Workflow using LangGraph in Code:

What is LangGraph

langGraph is product of Langchain. It is used to represent ai workflow by graph. Without graph/langGraph write logic of multi-agents ai-workflow is quite difficult.



Agent and Tool both can access State data and langgraph manage/update state.

Nodes: Represent specific actions or logic steps (e.g., calling an LLM, querying a database, or running a code snippet). Start & End are built-in nodes, to start and stop ai-workflow.

Edges: Determine the control flow or the path taken between nodes based on the output of previous steps.

Conditional Edges: Conditional edges are the decision-making components in LangGraph that dynamically route an application's control flow based on the current state. Unlike standard edges which always point to the exact same downstream node, conditional edges execute custom Python logic to determine which path to take next.

State: A shared, persistent data structure (like a conversation history) that passes information between nodes throughout the agent's entire execution

When to Use Which?

>> Use **LangChain** for straightforward tasks like basic chatbots, document parsing, or standard Retrieval-Augmented Generation (RAG) systems.

>> Use **LangGraph** for autonomous AI applications requiring memory, long-running loops, multi-step reasoning, or collaboration between multiple different AI agents.

Code Implement of AI-Workflow using LangGraph:

npm install @langchain/langgraph @langchain/core // if @langchain/core already installed remove it here because no need to reinstall it.

flow: create state/use built-in 'MessagesAnnotation' ----- create graph ----- create function & tools

Level4/index.js

```
import express from "express";
import dotenv from "dotenv";
import { ChatGroq } from "@langchain/groq";
import { Annotation, START, StateGraph } from "@langchain/langgraph";

dotenv.config();

const app = express();
app.use(express.json());
const port = process.env.PORT || 4000;

const llm = new ChatGroq({
  model: "openai/gpt-oss-120b",
  temperature: 0.7, //creativity of model-response: 0-> strict response, tepm>0 -
> creativity & long response
  maxTokens: 100, //generate response with maxtoken range.
  maxRetries: 2,
});

// implementation of langGraph
// Syntax to create State
const state = Annotation.Root({
  prompt: Annotation,
  aiMsg: Annotation,
});

//create function for graph
const callLLM = async (state) => {
  const response = await llm.invoke([
    {
      role: "system",
      content: "you are ai asistent",
    },
    {
      role: "user",
      content: state.prompt,
    },
  ]),
```

```

]);

return { aiMsg: response.content };
};

//create graph
const graph = new StateGraph(state)
  .addNode("agent", callLLM)
  .addEdge("__start__", "agent")
  .addEdge("agent", "__end__")
  .compile();

// with langchain CHATBOT
app.post("/chat", async (req, res) => {
  const { input } = req.body;

  const response = await graph.invoke({ prompt: input });
  res.status(200).json({ "ai: ": response });
});

app.listen(port, () => {
  console.log(`Server is Listen on ${port}`);
});

```

2.1 Create & Add Tools with LLM to build ai-agent:

Now add tools in llm to expand its functionality and make it ai-agent.

MessageAnnotation: langchain ki built-in state hoti h jis ma hm sra jo response h ya input h, osy ik 'messages' array k andr rakhty hn. so is k andr ik messages array hoti h jis k andr sari chzein rakhi hoti hn.

Tavily Search: famous web scraping tool. Now we use 'tavily serch langchain'.

https://docs.langchain.com/oss/javascript/integrations/tools/tavily_search

npm install @langchain/tavily

create tavily api-key: <https://app.tavily.com/home>

TAVILY_API_KEY=lkdjoifewjle

level4/index.js //import & config TavilySearch then add this new tool in tools[] array for tools config.
Use MessagesAnnotation 'state' which is langgraph built-in state for conditionalEdges.

```

import express from "express";
import dotenv from "dotenv";
import { ChatGroq } from "@langchain/groq";

```

```

import {
  Annotation,
  MessagesAnnotation,
  START,
  StateGraph,
} from "@langchain/langgraph";
import { ToolNode } from "@langchain/langgraph/prebuilt";
import { TavilySearch } from "@langchain/tavily";

dotenv.config();

const app = express();
app.use(express.json());
const port = process.env.PORT || 4000;

// implementation of langGraph
// create new tool [ tavilyhserch pkg that webserch if llm not have data about
query]
const tool = new TavilySearch({
  maxResults: 5,
  topic: "general",
});

const tools = [tool];
const toolNode = new ToolNode(tools); //runs tools

// setup llm
const llm = new ChatGroq({
  model: "openai/gpt-oss-120b",
  temperature: 0.7, //creativity of model-response: 0-> strict response, tepm>0 -
> creativity & long response
  maxTokens: 100, //generate response with maxtoken range.
  maxRetries: 2,
}).bindTools(tools); //so llm can use tools

// function that cell llm
const callLLM = async (state) => {
  const response = await llm.invoke([
    {
      role: "system",
      content: "you are ai asistent",
    },
    ...state.messages,
  ]);
};

```

```

    return { messages: [response] };
  });

// function for conditionalEdges -agent-shouldcontinue
const shouldContinue = async (state) => {
  const lastMessage = state.messages[state.messages.length - 1];
  if (lastMessage.tool_calls.length > 0) {
    return "tools";
  } else {
    return "__end__";
  }
};

// create graph for ai workflow - using langgraph
const graph = new StateGraph(MessagesAnnotation) //built-in state of langchain
  .addNode("agent", callLLM) // nodename 'agent' jo 'callLLM' function ko call
  kry ga
  .addNode("tools", toolNode)
  .addEdge("__start__", "agent") // edges bnai or start node ko agent node sy
  connect kr dia
  .addEdge("tools", "agent")
  .addConditionalEdges("agent", shouldContinue) //agent node sy 'shouldContinue'
  func run hoga to check kry ga 'tool call' ya 'end node'
  .compile();

// with langchain CHATBOT
app.post("/chat", async (req, res) => {
  const { input } = req.body;

  const response = await graph.invoke({
    messages: [{ role: "user", content: input }],
  });
  res
    .status(200)
    // .json({ "ai: ": response.messages[response.messages.length - 1].content
  });
  .json({ response });
});

app.listen(port, () => {
  console.log(`Server is Listen on ${port}`);
});

```


2.2 Create & Implement Memory for Our AI Agent

Stateless agent (no memory): Every request is independent; the model sees only the current prompt and forgets everything after responding.

Stateful agent (have and manage memory at each state) : The agent has a persistent memory/state store (DB, vector DB, Redis, LangGraph state, etc.) where it keeps context, past actions, user preferences, intermediate results, and task progress, and reuses them on the next step or conversation.

Implementation:

We have 2 options to add memory for llm,

1. Using redis
 2. Using langgraph built-in memory (MemorySaver from “@langchain/langgraph”)
- for now we use langgraph built-in memory, later in project we use redis for memory.

Thread Id: hr ik conversation (new chat) k liay ik id create hoti h jis sy hr ik specific conversation ki memory alg sy save hoti h. so hr ik user & conversation k liay different thread id create hoti h. But for now we hard code single thread id & later in project we learn how to create multiple thread ids.

Level4/Index.js //add MemorySaver() & config it in langgraph & hardcode threadId in graph.invoke

```
import express from "express";
import dotenv from "dotenv";
import { ChatGroq } from "@langchain/groq";
import {
  Annotation,
  MemorySaver,
  MessagesAnnotation,
  START,
  StateGraph,
} from "@langchain/langgraph";
import { ToolNode } from "@langchain/langgraph/prebuilt";
import { TavilySearch } from "@langchain/tavily";

dotenv.config();

const app = express();
app.use(express.json());
const port = process.env.PORT || 4000;

// implementation of langGraph
// create new tool [ tavilyhserch pkg that webserch if llm not have data about
query]
```

```

const tool = new TavilySearch({
  maxResults: 5,
  topic: "general",
});

const checkPointer = new MemorySaver(); //import {MemorySaver} from
"@langchain/langgraph";

const tools = [tool];
const toolNode = new ToolNode(tools); //runs tools

// setup llm
const llm = new ChatGroq({
  model: "openai/gpt-oss-120b",
  temperature: 0.7, //creativity of model-response: 0-> strict response, tepm>0 -
> creativity & long response
  maxTokens: 100, //generate response with maxtoken range.
  maxRetries: 2,
}).bindTools(tools); //so llm can use tools

// function that call llm
const callLLM = async (state) => {
  const response = await llm.invoke([
    { //note: system -prompt is very important for llm understanding what to do.
      role: "system",
      content: `Your are Jarvis AI assistant, user conversation memory first.

      Only use tools when the answer requires external real-time information
      like: weather, news, web search, stock prices etc.

      Do NOT call tools for simple conversation, memory-based questions,
      greetings, or personal context.`
    },
    ...state.messages,
  ]);

  return { messages: [response] };
};

// function for conditionalEdges -agent-shouldcontinue
const shouldContinue = async (state) => {
  const lastMessage = state.messages[state.messages.length - 1];
  if (lastMessage.tool_calls.length > 0) {
    return "tools";
  } else {

```

```

    return "__end__";
  }
};

// create graph for ai workflow - using langgraph
const graph = new StateGraph(MessagesAnnotation) //built-in state of langchain
  .addNode("agent", callLLM) // nodename 'agent' jo 'callLLM' function ko call
  kry ga
  .addNode("tools", toolNode)
  .addEdge("__start__", "agent") // edges bnai or start node ko agent node sy
  connect kr dia
  .addEdge("tools", "agent")
  .addConditionalEdges("agent", shouldContinue) //agent node sy 'shouldContinue'
  func run hoga to check kry ga 'tool call' ya 'end node'
  .compile({ checkpointer: checkPointer });

// with langchain CHATBOT
app.post("/chat", async (req, res) => {
  const { input } = req.body;

  const response = await graph.invoke(
    {
      messages: [{ role: "user", content: input }],
    },
    {
      configurable: { thread_id: "user123" }, //kuxh bhe thread id likh skty ho.
    },
  );
  res
    .status(200)
    .json({ ai: response.messages[response.messages.length - 1].content });
});

app.get("/", (req, res) => {
  res.status(200).json({ message: "helo from server" });
});

app.listen(port, () => {
  console.log(`Server is Listen on ${port}`);
});

```

Note: hm ny koi chezein hawa ma nhe chorien... ya sb implementation documentation ma likhi hoi hn.

<https://docs.langchain.com/oss/javascript/langgraph/checkpointers> is doc pr implemnetation likhi hoi h. apko docs parhny or smjhny any chaheya, doc pr kr onky code dry run kr k dekha krien.

Test ai-agent memory:

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:5000/ai
- Body:** JSON format containing:

```
1 {  
2   "input": "what is my name?"  
3 }
```
- Status:** 200 OK
- Size:** 29 Bytes
- Time:** 295 ms
- Response Body:**

```
1 {  
2   "ai": "Your name is Ayush."  
3 }
```